

Atelier OmniCore

Enterprise Technical Specification Document *Developed by IR Atelier | Version: 2026.1 Final*

1. System Architecture & Technology Stack

Atelier OmniCore is a secure, multi-tenant SaaS omnichannel helpdesk and AI customer support platform. It is designed to be highly cost-effective at launch while scaling to enterprise volumes.

Required Technology Stack

- Frontend (Dashboard & Widget):** React.js (Next.js) with Tailwind CSS. The public chat widget must be compiled into a single lightweight vanilla JS or Preact bundle.
- Backend API:** Node.js (Express/NestJS) or Python (FastAPI).
- Real-Time Engine:** Socket.io or native WebSockets.
- Database (Text/Data):** PostgreSQL with `pgvector` extension (e.g., Supabase or Neon.tech).
- Object Storage (Media/Files):** Cloudflare R2 (Provides 10GB free tier, zero egress fees).
- Background Jobs:** Redis (using BullMQ or Celery) for Web Crawling, PDF Generation, and Email dispatch.
- LLM Provider:** Gemini 1.5 Flash (via Google AI Studio for high-speed, free-tier processing).

2. Multi-Tenant Relational Schema (PostgreSQL)

The database strictly enforces isolation using `tenant_id` and sub-isolation using `brand_id`.

Core Tables

Table	Primary Key	Foreign Keys	Key Columns & Indexes
Tenants	<code>id</code> (UUID)	None	<code>company_name</code> , <code>subscription_status</code> , <code>lemon_squeezy_customer_id</code>
Brands	<code>id</code> (UUID)	<code>tenant_id</code>	<code>brand_name</code> , <code>widget_config_json</code> , <code>allowed_domains_array</code> (CORS), <code>inbound_email_prefix</code>
Agents	<code>id</code> (UUID)	<code>tenant_id</code>	<code>name</code> , <code>role</code> (admin/agent), <code>brand_access_array</code> , <code>personal_settings_json</code>
Visitors	<code>id</code> (UUID)	<code>tenant_id</code> , <code>brand_id</code>	<code>session_token</code> , <code>email</code> , <code>last_seen_at</code> , <code>ip_address</code> , <code>location_city</code>
Conversations	<code>id</code> (UUID)	<code>tenant_id</code> , <code>brand_id</code> , <code>visitor_id</code> , <code>assigned_agent_id</code>	<code>status</code> (ai_handling, open, closed), <code>priority</code> , <code>csat_score</code> , <code>sla_breach_at</code>

Messages	<code>id</code> (UUID)	<code>conversation_id</code>	<code>sender_type</code> , <code>message_body</code> , <code>is_internal_note</code> , <code>attachments_json</code>
-----------------	---------------------------	------------------------------	--

Knowledge Base & Help Center (pgvector)

- **Knowledge_Articles Table:** `id` , `tenant_id` , `brand_id` , `title` , `public_html_content` (for Help Center rendering), `is_public` (Boolean).
- **AI_Embeddings Table:** `id` , `article_id` (FK), `source_url` (if scraped), `chunked_text` , `embedding_vector` (VECTOR 1536).

3. Storage & Export Architecture (Scalability)

Data vs. Media Separation

- **Chat Text:** Stored directly in PostgreSQL.
- **Rich Media (Screenshots/PDFs):** The chat widget uploads attachments directly to **Cloudflare R2** via Pre-Signed URLs. The backend database only stores the secure `https://...` URL in the `attachments_json` column. Max file size: 10MB.
- **PDF / TXT Export Engine:** Agents or Visitors can click "Export Thread". A Redis background worker fetches the conversation history, uses a library like `Puppeteer` or `pdfkit` to generate a styled PDF, uploads it to R2, and returns a temporary download link to the user.

4. Configuration & Settings Hierarchy

Tier 1: Global Tenant Settings (The Agency)

- **Billing & Subscriptions:** Managed via Lemon Squeezy Customer Portal.
- **API Integrations:** Master keys for LLMs (Gemini API) and Email routing.

Tier 2: Brand-Level Settings (Isolated Entities)

- **Widget Security & Customization:** Colors, logo, launcher icon. Security Requirement: `allowed_domains_array` enforces strict CORS validation.
- **AI Response Config:** System Prompt (Tone of voice), Confidence Threshold.
- **Public Help Center:** Custom CNAME routing logic (e.g., `help.clientbrand.com`).

Tier 3: Agent/Staff Settings

- **Profile & Preferences:** Display name, avatar, email signature, UI Theme, and notification toggles.
- **Smart Views:** Custom inbox filters (e.g., "Show only Brand X tickets older than 10 mins").

5. Real-Time Telemetry & Socket Architecture

Connection Resilience & Offline Queueing

- **WebSockets:** Handles mobile connectivity drops. If the socket disconnects, the client-side script queues outgoing messages locally and re-syncs upon reconnection.
- **Standardized Socket Events:** `client:telemetry_update` , `client:send_message` .

- **Collision Detection:** Broadcasts `agent:is_typing` events to prevent two agents from replying to the same ticket simultaneously.

6. Cost-Effective Email Engine & Ticket Routing

Inbound / Outbound Strategy (SendGrid or Resend)

- **Outbound Transcripts & Replies:** Routed via Resend/SendGrid REST APIs. Uses a shared root domain (e.g., `mail.iratelier.com`) to avoid per-domain verification fees for free-tier clients. Includes CSAT surveys on ticket close.
- **Inbound Mail Parsing (Webhook):**
 1. Clients are assigned a unique routing address (e.g., `support@[tenant_prefix].iratelier.com`).
 2. When a customer replies, the email provider's Inbound Parse Webhook posts the raw email to `/api/webhooks/inbound-mail`.
 3. The backend parses the JSON, identifies the ticket via `In-Reply-To` headers, and appends the message.
- **Quote Stripping Engine:** Uses Regex libraries to strip previous email chain quotes so the agent only sees the *new* text.

Agent Productivity Tools

- **SLA Timers:** Auto-calculates "First Response Time". Escalates tickets to managers if SLA breaches.
- **Macros:** Agents type `/` to instantly insert rich-text responses.
- **Internal Notes:** Agents can leave private messages in the thread (yellow background) invisible to the customer.

7. Dual-Layer AI Engine (Powered by Gemini 1.5 Flash)

Data Ingestion Engine (How the AI Learns)

1. **Web Crawler:** A Redis worker crawls the client's sitemap, extracts `<p>` tags, chunks the text, and generates embeddings into the Vector database.
2. **Manual Article Creation:** Staff write articles. When published, an automated job vectorizes the text.

Layer 1: Inbound RAG Core (First Responder)

- **Brand-Specific Vector Search:** Queries strictly by `brand_id` so the AI never leaks another brand's data.
- **Human Handover:** If the AI cannot answer confidently based *only* on the vector context, it triggers a handover protocol changing the ticket status to `open` and notifying agents.

Layer 2: In-Context Embedded Agent Copilot

- **Rephrase Engine:** Agent draft text is sent to Gemini to rewrite for clarity and tone.
- **Smart Summarize:** Background task generates a private summary of a long chat thread when a human takes over from the bot.

8. Billing & Monetization (Lemon Squeezy)

The platform uses **Lemon Squeezy** as the Merchant of Record.

1. **Checkout Flow:** Users upgrade via a Lemon Squeezy hosted checkout link containing `custom_data[tenant_id]` .
2. **Webhook Listener (/api/webhooks/lemonsqueezy):**
 - Listens for `subscription_created` , `subscription_updated` , and `subscription_cancelled` events.
 - Verifies the webhook signature using the Lemon Squeezy Webhook Secret.
 - Updates the `subscription_status` in the `Tenants` table automatically.
3. **Grace Period Logic:** If a payment fails, status changes to `past_due` , triggering a 3-day grace period before public widgets are disabled.

9. Zero-Trust Security Specification

- **API Gateway Layer:** Strict rate limiting and JWT Auth Verification.
- **Application Core Layer:** Explicit Content Security Policy (CSP) blocking unauthorized script execution on client sites.
- **Input Hygiene Layer:** HTML sanitization and SQL parameterization to prevent injection.
- **Orchestration Layer:** Context-isolated database connection pools per tenant (enforced `tenant_id` WHERE clauses on all queries).
- **Storage Encryption Layer:** Argon2id algorithms for password hashing.